保険約款ドラフト作成アプリケーション 詳細技術設計書

プロジェクト概要

設計方針

- **メンテナンス性**: モジュラー設計、明確な責任分離、包括的なテスト
- **拡張性**: プラグイン機構、マイクロサービス、API-First設計
- **商用利用**: 独自ブランディング、ライセンス適合性、知的財産権保護

非機能要件

- **可用性**: 99.9%以上のアップタイム
- **パフォーマンス**: レスポンス時間3秒以内
- **スケーラビリティ**: 同時ユーザー1000人対応
- **セキュリティ**: SOC2 Type II準拠
- **保守性**: コードカバレッジ80%以上

TODO管理

- [x] Phase 1: 技術スタック詳細選定
- [x] フロントエンド技術選定
- [x] バックエンド技術選定
- [x] データベース技術選定
- [x] AI統合技術選定
- [x] インフラ・デプロイ技術選定
- [x] Phase 2: システムアーキテクチャ設計
- [x] 全体アーキテクチャ設計
- [x] マイクロサービス分割
- [x] API設計方針
- [x] セキュリティアーキテクチャ
- [x] Phase 3: データベース・API設計

- [x] データモデル設計
- [x] API仕様設計

- [x] 認証・認可設計

- [x] Phase 4: フロントエンド・UI設計

- [x] コンポーネント設計
- [x] 状態管理設計

- [x] UI/UX設計

- [x] Phase 5: 拡張性・保守性設計

- [x] プラグイン機構設計
- [x] 監視・ログ設計

- [x] CI/CD設計

- [x] Phase 6: 実装ガイド・設計書作成

- [x] 開発ガイドライン
- [x] 運用手順書
- [x] 最終設計書

Phase 1: 技術スタック詳細選定

1.1 フロントエンド技術スタック

推奨構成：React + TypeScript + Monaco Editor

React 18.3+ の選定理由: - **メンテナンス性**: - 豊富なエコシステムと長期サポート - 大規模なコミュニティとドキュメント - 成熟したテストツール（Jest, React Testing Library） - **拡張性**: - コンポーネントベース設計 - Concurrent Features による性能向上 - Server Components 対応（将来的なSSR）

TypeScript 5.0+ の選定理由: - **メンテナンス性**: - 静的型チェックによるバグ早期発見 - IDEサポートによる開発効率向上 - リファクタリング安全性 - **拡張性**: - 型安全なAPI設計 - 大規模チーム開発対応 - 段階的導入可能

Monaco Editor 0.52+ の選定理由: - **機能性**: - VSCodeと同等のエディター機能 - 豊富な言語サポート - カスタマイズ可能なテーマ・拡張 - **ライセンス**: MIT License（商用利用フレンドリー） - **保険約款特化カスタマイズ**: - 約款専用シンタックスハイライト - 条項番号自動採番 - 参考約款との差分表示

状態管理：Zustand + React Query

Zustand の選定理由: - **シンプルさ**: Redux比で70%少ないボイラープレート - **TypeScript親和性**: 型安全な状態管理 - **メンテナンス性**: 小さなバンドルサイズ（2.9KB） - **拡張性**: ミドルウェア対応

React Query (TanStack Query) の選定理由: - **サーバー状態管理**: キャッシュ・同期・更新の自動化 - **パフォーマンス**: 自動的な背景更新・重複リクエスト排除 - **開発体験**: DevTools による状態可視化

UI コンポーネント：Mantine + Tailwind CSS

Mantine の選定理由: - **メンテナンス性**: TypeScript-first設計 - **アクセシビリティ**: WCAG 2.1 AA準拠 - **カスタマイズ性**: CSS-in-JS + CSS Variables - **豊富なコンポーネント**: 100+ コンポーネント

Tailwind CSS の選定理由: - **保守性**: Utility-first による一貫性 - **パフォーマンス**: 未使用CSS自動削除 - **開発速度**: 高速プロトタイピング

1.2 バックエンド技術スタック

推奨構成：Node.js + NestJS + TypeScript

Node.js 20 LTS の選定理由: - **パフォーマンス**: V8エンジン最適化 - **エコシステム**: npm パッケージ豊富 - **開発効率**: フロントエンドとの言語統一 - **長期サポート**: 2026年4月まで

NestJS 10+ の選定理由: - **メンテナンス性**: - Angular風のデコレーター設計 - 依存性注入による疎結合 - 自動生成されるOpenAPI仕様 - **拡張性**: - モジュラーアーキテクチャ - マイクロサービス対応 - GraphQL/REST両対応 - **エンタープライズ対応**: - 認証・認可機能内蔵 - バリデーション・シリアライゼーション - 包括的なテスト機能

API設計：GraphQL + REST ハイブリッド

GraphQL の採用領域: - 複雑なデータ取得（約款+メタデータ+履歴） - リアルタイム更新（WebSocket + Subscription） - フロントエンド最適化（必要データのみ取得）

REST の採用領域: - ファイルアップロード/ダウンロード - 外部システム連携 - シンプルなCRUD操作

1.3 データベース技術スタック

推奨構成：PostgreSQL + Redis + MinIO

PostgreSQL 16+ の選定理由: - **拡張性**: - pgvector による ベクトル検索（RAG機能） - JSONB による柔軟なスキーマ - パーティショニング対応 - **メンテナンス性**: - ACID準拠の信頼性 - 豊富な運用ツール - 長期サポート

Redis 7+ の選定理由: - **パフォーマンス**: インメモリキャッシュ - **機能性**: セッション管理・レート制限・ジョブキュー - **拡張性**: Redis Cluster 対応

MinIO の選定理由: - **S3互換**: AWS移行容易性 - **オンプレミス対応**: データ主権確保 - **高可用性**: 分散ストレージ

1.4 AI統合技術スタック

推奨構成：LangChain + 複数LLMプロバイダー

LangChain の選定理由: - **抽象化**: 複数LLMプロバイダーの統一インターフェース - **RAG機能**: ベクトルストア・検索機能内蔵 - **拡張性**: カスタムチェーン・エージェント作成 - **メンテナンス性**: 豊富なドキュメント・コミュニティ

対応LLMプロバイダー: - **OpenAI**: GPT-4o（汎用・高品質） - **Anthropic**: Claude 3.5 Sonnet（長文・推論） - **Google**: Gemini Pro（多言語・コスト効率） - **ローカルLLM**: Ollama（プライバシー・コスト削減）

ベクトルデータベース：pgvector

pgvector の選定理由: - **統合性**: PostgreSQL拡張として運用簡素化 - **コスト効率**: 専用ベクトルDB不要 - **SQL互換**: 既存クエリとの組み合わせ - **スケーラビリティ**: PostgreSQLのスケーリング機能活用

1.5 インフラ・デプロイ技術スタック

推奨構成：Docker + Kubernetes + Terraform

Docker の選定理由: - **一貫性**: 開発・本番環境の統一 - **移植性**: マルチクラウド対応 - **効率性**: リソース使用量最適化

Kubernetes の選定理由: - **スケーラビリティ**: 自動スケーリング - **可用性**: 自動復旧・ローリングアップデート - **拡張性**: Helm Chart・Operator対応

Terraform の選定理由: - **Infrastructure as Code**: バージョン管理・再現性 - **マルチクラウド**: AWS・Azure・GCP対応 - **メンテナンス性**: 宣言的設定・差分管理

CI/CD：GitHub Actions + ArgoCD

GitHub Actions の選定理由: - **統合性**: ソースコード管理との一体化 - **柔軟性**: カスタムワークフロー - **コスト効率**: パブリックリポジトリ無料

ArgoCD の選定理由: - **GitOps**: 宣言的デプロイメント - **可視性**: デプロイ状況の可視化 - **安全性**: Pull-based デプロイメント

1.6 監視・ログ技術スタック

推奨構成：Prometheus + Grafana + ELK Stack

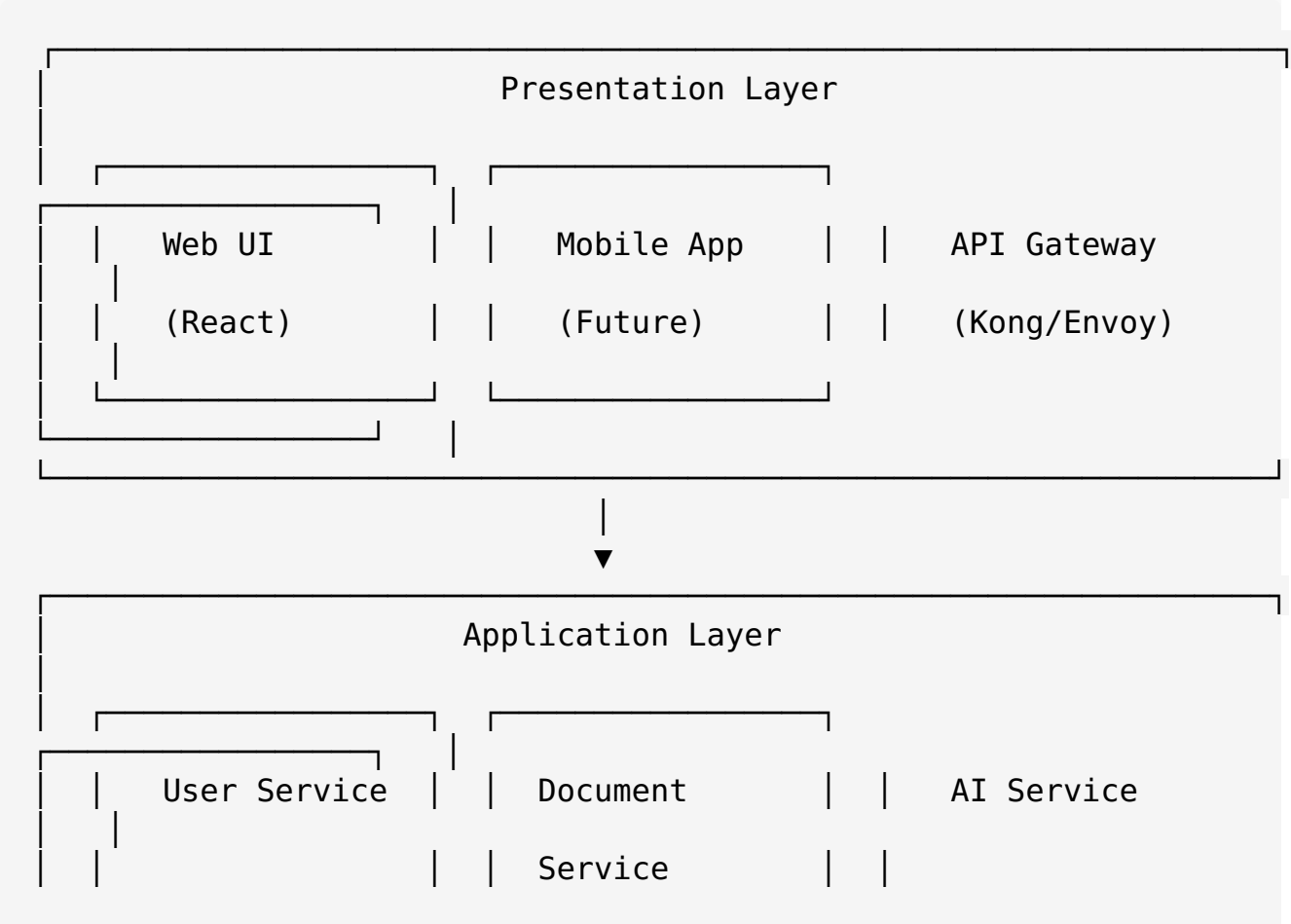
Prometheus + Grafana: - **メトリクス監視**: アプリケーション・インフラ - **アラート**: 閾値ベース・異常検知 - **可視化**: ダッシュボード・レポート

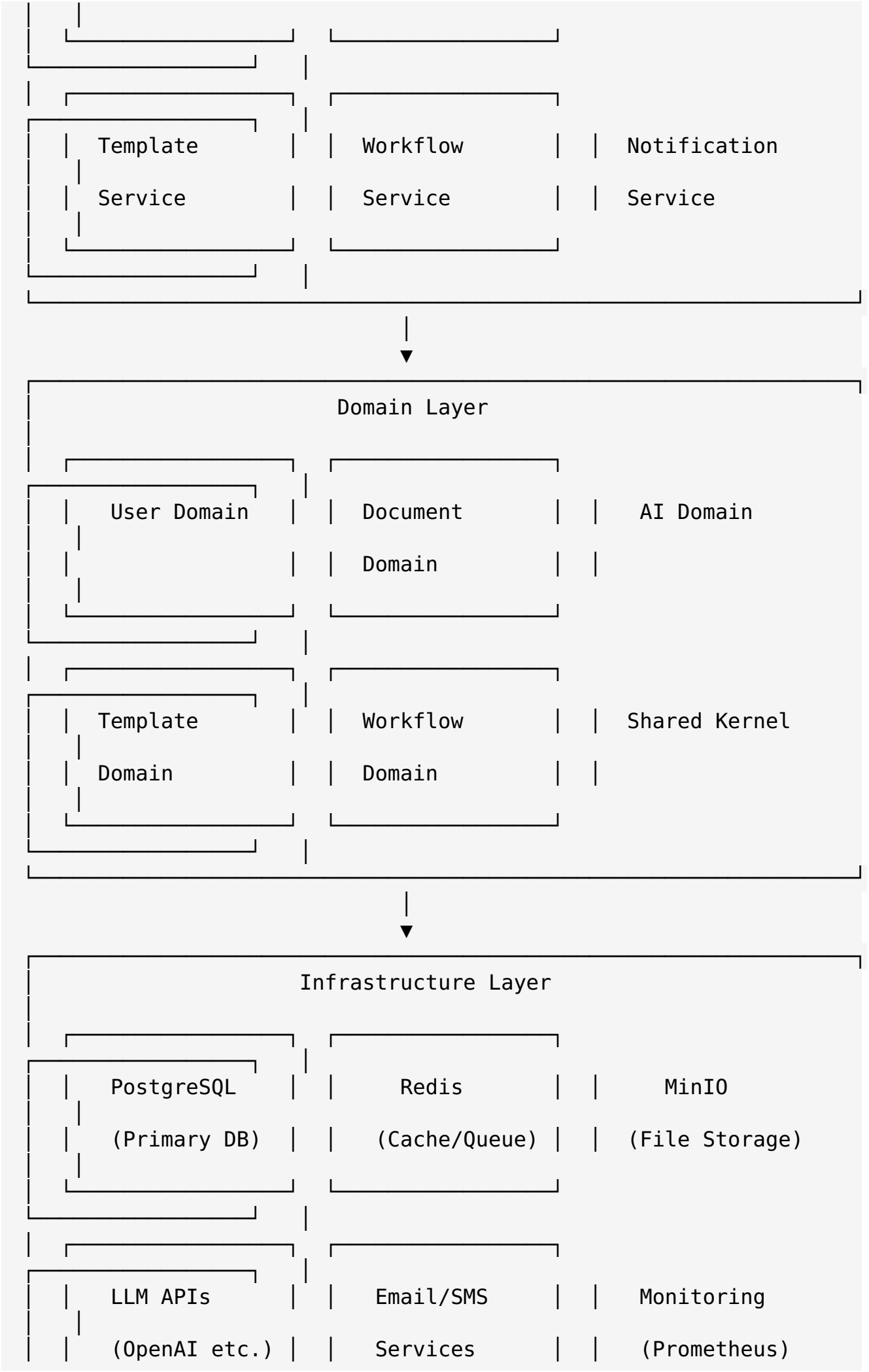
ELK Stack (Elasticsearch + Logstash + Kibana): - **ログ集約**: 構造化ログ・全文検索 - **分析**: ユーザー行動・エラー分析 - **可視化**: ログダッシュボード

Phase 2: システムアーキテクチャ設計

2.1 全体アーキテクチャ概要

ヘキサゴナルアーキテクチャ（ポート&アダプター）の採用




Domain Layer
Template Service
Workflow Service
Notification Service
User Domain
Document Domain
AI Domain
Template Domain
Workflow Domain
Shared Kernel
Infrastructure Layer
PostgreSQL (Primary DB)
Redis (Cache/Queue)
MinIO (File Storage)
LLM APIs (OpenAI etc.)
Email/SMS Services
Monitoring (Prometheus)

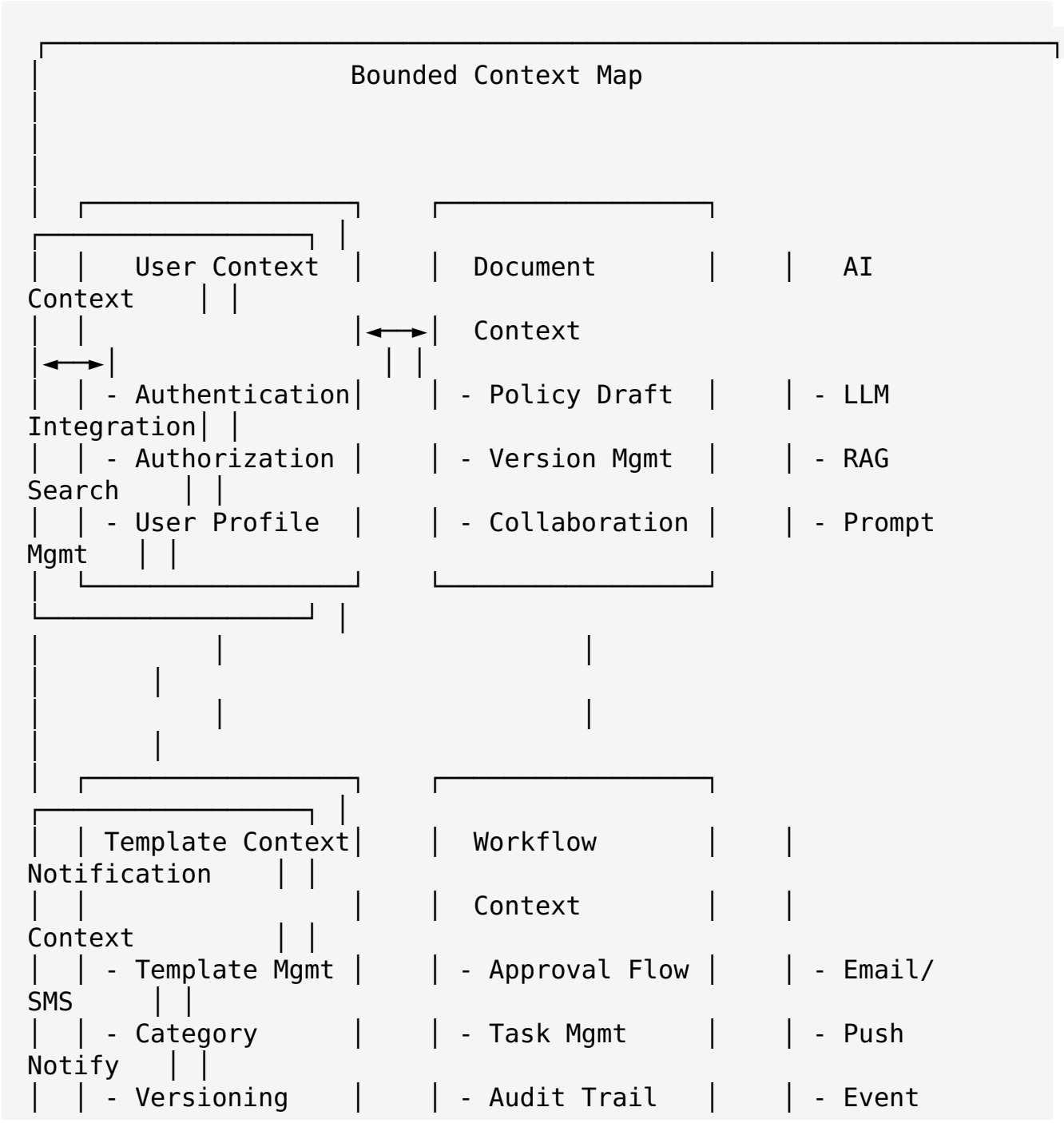
```
│    │
│    └──────────────────┘    └──────────────────┘
└──────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

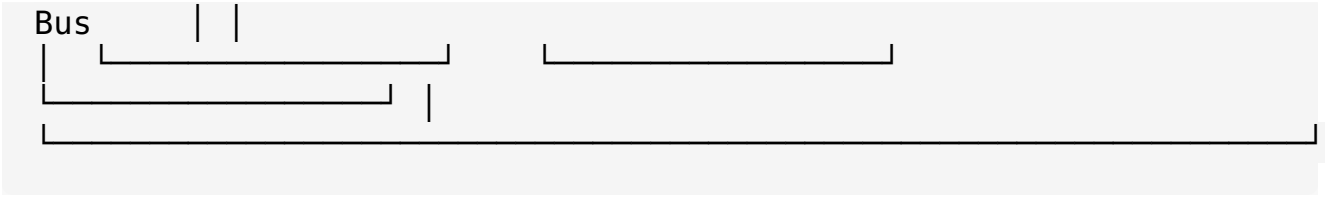
アーキテクチャ選定理由

ヘキサゴナルアーキテクチャの利点: - **メンテナンス性**: ビジネスロジックとインフラの分離 - **テスタビリティ**: モック・スタブによる単体テスト容易性 - **拡張性**: 新しいアダプターの追加が容易 - **技術負債軽減**: 外部依存の変更影響を局所化

2.2 マイクロサービス分割戦略

ドメイン駆動設計（DDD）による境界コンテキスト





サービス分割詳細

1. User Service（ユーザー管理サービス） - **責務**: 認証・認可・ユーザープロファイル管理 - **技術**: NestJS + Passport + JWT - **データストア**: PostgreSQL - **API**: REST + GraphQL

2. Document Service（文書管理サービス） - **責務**: 約款ドラフト作成・編集・バージョン管理 - **技術**: NestJS + TypeORM + WebSocket - **データストア**: PostgreSQL + MinIO - **API**: GraphQL + WebSocket

3. AI Service（AI統合サービス） - **責務**: LLM統合・RAG検索・プロンプト管理 - **技術**: NestJS + LangChain + pgvector - **データストア**: PostgreSQL (pgvector) + Redis - **API**: GraphQL + Server-Sent Events

4. Template Service（テンプレート管理サービス） - **責務**: 約款テンプレート・カテゴリ管理 - **技術**: NestJS + TypeORM - **データストア**: PostgreSQL - **API**: REST + GraphQL

5. Workflow Service（ワークフロー管理サービス） - **責務**: 承認フロー・タスク管理・監査ログ - **技術**: NestJS + Bull Queue + Redis - **データストア**: PostgreSQL + Redis - **API**: GraphQL + WebSocket

6. Notification Service（通知サービス） - **責務**: メール・SMS・プッシュ通知 - **技術**: NestJS + Bull Queue + Redis - **データストア**: Redis - **API**: REST

2.3 API設計方針

API-First設計の採用

OpenAPI 3.1 仕様書駆動開発: - **設計フェーズ**: API仕様書作成 → レビュー → 承認 - **開発フェーズ**: モック生成 → 並行開発 → 統合テスト - **運用フェーズ**: 自動ドキュメント生成 → バージョン管理

GraphQL + REST ハイブリッド戦略

GraphQL採用領域:

```
# 複雑なデータ取得
type Query {
  policyDraft(id: ID!): PolicyDraft
  searchPolicies(filter: PolicyFilter): [PolicyDraft]
  userProfile: User
```

```
}

# リアルタイム更新
type Subscription {
  policyDraftUpdated(id: ID!): PolicyDraft
  collaborationEvents(documentId: ID!): CollaborationEvent
}

# 複雑な操作
type Mutation {
  createPolicyDraft(input: CreatePolicyDraftInput!): PolicyDraft
  generateAIContent(input: AIGenerationInput!):
AIGenerationResult
}
```

REST採用領域:

```
// ファイル操作
POST   /api/v1/documents/{id}/upload
GET    /api/v1/documents/{id}/download
DELETE /api/v1/documents/{id}/attachments/{attachmentId}

// 外部システム連携
POST   /api/v1/integrations/webhook
GET    /api/v1/integrations/status

// ヘルスチェック・メトリクス
GET    /api/v1/health
GET    /api/v1/metrics
```

2.4 セキュリティアーキテクチャ

多層防御戦略

```
┌─────────────────────────────────────────────────────────────────────┐
│                        Security Layers                              │
│                                                                     │
│                                                                     │
│                                                                     │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
┌─────────────────────────────────────────────────────────────────────┐
│                                                                     │
│   │                    Network Security                             │
│  │                                                                  │
│   │  • WAF (Web Application Firewall)                               │
│  │                                                                  │
│   │  • DDoS Protection                                              │
│  │                                                                  │
```


• TLS 1.3 Encryption
Application Security
• OAuth 2.0 + OIDC Authentication
• RBAC (Role-Based Access Control)
• API Rate Limiting
• Input Validation & Sanitization
Data Security
• Encryption at Rest (AES-256)
• Encryption in Transit (TLS 1.3)
• Database Access Control
• PII Data Masking
Infrastructure Security
• Container Security Scanning
• Kubernetes RBAC
• Network Policies
• Secret Management (Vault)

```
└──────────────────────────────────────────────────────────────────────┘
```

認証・認可設計

OAuth 2.0 + OpenID Connect:

```typescript
// JWT Token Structure
interface JWTPayload {
  sub: string;            // User ID
  email: string;          // User Email
  roles: string[];        // User Roles
  permissions: string[]; // Fine-grained Permissions
  org: string;           // Organization ID
  iat: number;           // Issued At
  exp: number;           // Expiration
}

// RBAC Model
interface Role {
  id: string;
  name: string;
  permissions: Permission[];
  organization: string;
}

interface Permission {
  resource: string;       // e.g., "policy_draft"
  action: string;         // e.g., "read", "write", "delete"
  conditions?: object;    // e.g., {"owner": true}
}
```

2.5 データー貫性・トランザクション戦略

Saga パターンによる分散トランザクション

```typescript
// Policy Draft Creation Saga
class PolicyDraftCreationSaga {
  async execute(command: CreatePolicyDraftCommand) {
    const saga = new SagaTransaction();

    try {
      // Step 1: Create draft document
      const draft = await saga.step(
        () =>
this.documentService.createDraft(command.documentData),
        (draft) => this.documentService.deleteDraft(draft.id)
      );
```

```
      // Step 2: Initialize AI context
      const aiContext = await saga.step(
        () => this.aiService.initializeContext(draft.id,
command.template),
        (context) => this.aiService.deleteContext(context.id)
      );

      // Step 3: Create workflow
      const workflow = await saga.step(
        () => this.workflowService.createWorkflow(draft.id,
command.workflow),
        (workflow) =>
this.workflowService.deleteWorkflow(workflow.id)
      );

      await saga.commit();
      return { draft, aiContext, workflow };

    } catch (error) {
      await saga.rollback();
      throw error;
    }
  }
}
```

イベント駆動アーキテクチャ

```
// Domain Events
interface DomainEvent {
  id: string;
  aggregateId: string;
  eventType: string;
  payload: object;
  timestamp: Date;
  version: number;
}

// Event Bus Implementation
class EventBus {
  async publish(event: DomainEvent): Promise<void> {
    // Publish to Redis Streams
    await this.redis.xadd(
      `events:${event.eventType}`,
      '*',
      'data', JSON.stringify(event)
    );
  }

  async subscribe(eventType: string, handler: EventHandler):
```

```
Promise<void> {
    // Subscribe to Redis Streams
    const stream = `events:${eventType}`;
    // Implementation details...
  }
}
```

Phase 3: データベース・API設計

3.1 データモデル設計

エンティティ関係図（ERD）

```
 -- Core Entities
CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    domain VARCHAR(255) UNIQUE,
    settings JSONB DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id),
    email VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL DEFAULT 'user',
    permissions JSONB DEFAULT '[]',
    profile JSONB DEFAULT '{}',
    is_active BOOLEAN DEFAULT true,
    last_login_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Document Management
CREATE TABLE policy_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id),
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100) NOT NULL,
    description TEXT,
    content JSONB NOT NULL, -- Structured template content
    metadata JSONB DEFAULT '{}',
    version INTEGER DEFAULT 1,
    is_active BOOLEAN DEFAULT true,
```

```
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE policy_drafts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id),
    template_id UUID REFERENCES policy_templates(id),
    title VARCHAR(255) NOT NULL,
    content JSONB NOT NULL, -- Rich text content with structure
    metadata JSONB DEFAULT '{}',
    status VARCHAR(50) DEFAULT 'draft', -- draft, review,
approved, published
    version INTEGER DEFAULT 1,
    parent_id UUID REFERENCES policy_drafts(id), -- For
versioning
    created_by UUID REFERENCES users(id),
    assigned_to UUID REFERENCES users(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- AI Integration
CREATE TABLE ai_contexts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_draft_id UUID REFERENCES policy_drafts(id),
    context_type VARCHAR(50) NOT NULL, -- 'generation',
'review', 'suggestion'
    prompt_template TEXT,
    context_data JSONB DEFAULT '{}',
    model_config JSONB DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE ai_interactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    context_id UUID REFERENCES ai_contexts(id),
    user_id UUID REFERENCES users(id),
    input_text TEXT NOT NULL,
    output_text TEXT,
    model_used VARCHAR(100),
    tokens_used INTEGER,
    processing_time_ms INTEGER,
    feedback_score INTEGER, -- 1-5 rating
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Vector Storage for RAG
CREATE TABLE document_embeddings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID,
```

```sql
-- Can reference policy_drafts or external docs
    document_type VARCHAR(50) NOT NULL,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536), -- OpenAI ada-002 dimension
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Workflow Management
CREATE TABLE workflows (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_draft_id UUID REFERENCES policy_drafts(id),
    name VARCHAR(255) NOT NULL,
    definition JSONB NOT NULL, -- Workflow steps and rules
    current_step VARCHAR(100),
    status VARCHAR(50) DEFAULT 'active',
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE workflow_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID REFERENCES workflows(id),
    assignee_id UUID REFERENCES users(id),
    task_type VARCHAR(100) NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    status VARCHAR(50) DEFAULT 'pending',
    due_date TIMESTAMP WITH TIME ZONE,
    completed_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Audit and History
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type VARCHAR(100) NOT NULL,
    entity_id UUID NOT NULL,
    action VARCHAR(100) NOT NULL,
    user_id UUID REFERENCES users(id),
    changes JSONB,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Indexes for Performance
CREATE INDEX idx_policy_drafts_org_status ON
policy_drafts(organization_id, status);
CREATE INDEX idx_policy_drafts_created_by ON
```

```
policy_drafts(created_by);
CREATE INDEX idx_document_embeddings_vector ON
document_embeddings USING ivfflat (embedding vector_cosine_ops);
CREATE INDEX idx_ai_interactions_context ON
ai_interactions(context_id);
CREATE INDEX idx_workflow_tasks_assignee ON
workflow_tasks(assignee_id, status);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type,
entity_id);
```

ドメインモデル設計

```
// Domain Entities
export class PolicyDraft {
  constructor(
    public readonly id: PolicyDraftId,
    public readonly organizationId: OrganizationId,
    public title: string,
    public content: PolicyContent,
    public status: PolicyStatus,
    public metadata: PolicyMetadata,
    public readonly createdBy: UserId,
    public assignedTo?: UserId
  ) {}

  // Business Logic Methods
  public assignTo(userId: UserId, assignedBy: UserId): void {
    if (!this.canBeAssignedBy(assignedBy)) {
      throw new Error('User does not have permission to assign
this draft');
    }
    this.assignedTo = userId;
    this.addDomainEvent(new PolicyDraftAssignedEvent(this.id,
userId, assignedBy));
  }

  public updateContent(content: PolicyContent, updatedBy:
UserId): void {
    if (!this.canBeEditedBy(updatedBy)) {
      throw new Error('User does not have permission to edit
this draft');
    }
    this.content = content;
    this.addDomainEvent(new PolicyDraftUpdatedEvent(this.id,
content, updatedBy));
  }

  public approve(approvedBy: UserId): void {
    if (this.status !== PolicyStatus.REVIEW) {
      throw new Error('Draft must be in review status to be
```

```
approved');
    }
    this.status = PolicyStatus.APPROVED;
    this.addDomainEvent(new PolicyDraftApprovedEvent(this.id,
approvedBy));
  }

  private canBeAssignedBy(userId: UserId): boolean {
    // Business logic for assignment permissions
    return true; // Simplified
  }

  private canBeEditedBy(userId: UserId): boolean {
    // Business logic for edit permissions
    return this.createdBy.equals(userId) ||
this.assignedTo?.equals(userId);
  }
}

// Value Objects
export class PolicyContent {
  constructor(
    public readonly sections: PolicySection[],
    public readonly format: ContentFormat
  ) {
    this.validate();
  }

  private validate(): void {
    if (this.sections.length === 0) {
      throw new Error('Policy content must have at least one
section');
    }
  }

  public addSection(section: PolicySection): PolicyContent {
    return new PolicyContent([...this.sections, section],
this.format);
  }

  public updateSection(index: number, section: PolicySection):
PolicyContent {
    const newSections = [...this.sections];
    newSections[index] = section;
    return new PolicyContent(newSections, this.format);
  }
}

export class PolicySection {
  constructor(
    public readonly id: string,
    public readonly title: string,
```

```
    public readonly content: string,
    public readonly order: number,
    public readonly type: SectionType
  ) {}
}

// Aggregates
export class AIContext {
  constructor(
    public readonly id: AIContextId,
    public readonly policyDraftId: PolicyDraftId,
    public readonly contextType: AIContextType,
    private interactions: AIInteraction[] = []
  ) {}

  public addInteraction(
    input: string,
    userId: UserId,
    modelConfig: ModelConfig
  ): Promise<AIInteraction> {
    const interaction = new AIInteraction(
      AIInteractionId.generate(),
      this.id,
      userId,
      input,
      modelConfig
    );

    this.interactions.push(interaction);
    this.addDomainEvent(new
AIInteractionCreatedEvent(interaction));

    return Promise.resolve(interaction);
  }

  public getInteractionHistory(): ReadonlyArray<AIInteraction> {
    return [...this.interactions];
  }
}
```

3.2 API仕様設計

GraphQL Schema Definition

```
# Scalars
scalar DateTime
scalar JSON
scalar UUID

# Enums
```

```
enum PolicyStatus {
  DRAFT
  REVIEW
  APPROVED
  PUBLISHED
  ARCHIVED
}

enum UserRole {
  ADMIN
  MANAGER
  EDITOR
  VIEWER
}

enum AIModelType {
  GPT_4O
  CLAUDE_3_5_SONNET
  GEMINI_PRO
  LOCAL_LLM
}

# Types
type Organization {
  id: UUID!
  name: String!
  domain: String
  settings: JSON
  users: [User!]!
  policyTemplates: [PolicyTemplate!]!
  createdAt: DateTime!
  updatedAt: DateTime!
}

type User {
  id: UUID!
  organization: Organization!
  email: String!
  name: String!
  role: UserRole!
  permissions: [String!]!
  profile: JSON
  isActive: Boolean!
  lastLoginAt: DateTime
  createdPolicyDrafts: [PolicyDraft!]!
  assignedPolicyDrafts: [PolicyDraft!]!
  createdAt: DateTime!
  updatedAt: DateTime!
}

type PolicyTemplate {
  id: UUID!
```

```
  organization: Organization!
  name: String!
  category: String!
  description: String
  content: JSON!
  metadata: JSON
  version: Int!
  isActive: Boolean!
  createdBy: User!
  policyDrafts: [PolicyDraft!]!
  createdAt: DateTime!
  updatedAt: DateTime!
}

type PolicyDraft {
  id: UUID!
  organization: Organization!
  template: PolicyTemplate
  title: String!
  content: JSON!
  metadata: JSON
  status: PolicyStatus!
  version: Int!
  parent: PolicyDraft
  children: [PolicyDraft!]!
  createdBy: User!
  assignedTo: User
  aiContext: AIContext
  workflow: Workflow
  collaborators: [User!]!
  comments: [Comment!]!
  createdAt: DateTime!
  updatedAt: DateTime!
}

type AIContext {
  id: UUID!
  policyDraft: PolicyDraft!
  contextType: String!
  promptTemplate: String
  contextData: JSON
  modelConfig: JSON
  interactions: [AIInteraction!]!
  createdAt: DateTime!
}

type AIInteraction {
  id: UUID!
  context: AIContext!
  user: User!
  inputText: String!
  outputText: String
```

```
  modelUsed: AIModelType
  tokensUsed: Int
  processingTimeMs: Int
  feedbackScore: Int
  createdAt: DateTime!
}

type Workflow {
  id: UUID!
  policyDraft: PolicyDraft!
  name: String!
  definition: JSON!
  currentStep: String
  status: String!
  tasks: [WorkflowTask!]!
  createdBy: User!
  createdAt: DateTime!
  updatedAt: DateTime!
}

type WorkflowTask {
  id: UUID!
  workflow: Workflow!
  assignee: User!
  taskType: String!
  title: String!
  description: String
  status: String!
  dueDate: DateTime
  completedAt: DateTime
  createdAt: DateTime!
}

# Input Types
input CreatePolicyDraftInput {
  templateId: UUID
  title: String!
  content: JSON!
  metadata: JSON
  assignedTo: UUID
}

input UpdatePolicyDraftInput {
  title: String
  content: JSON
  metadata: JSON
  status: PolicyStatus
  assignedTo: UUID
}

input AIGenerationInput {
  policyDraftId: UUID!
```

```
  prompt: String!
  modelType: AIModelType!
  contextType: String!
  options: JSON
}

input PolicySearchFilter {
  organizationId: UUID
  status: [PolicyStatus!]
  createdBy: UUID
  assignedTo: UUID
  category: String
  dateRange: DateRangeInput
  textSearch: String
}

input DateRangeInput {
  from: DateTime!
  to: DateTime!
}

# Queries
type Query {
  # User Management
  me: User
  users(organizationId: UUID!): [User!]!
  user(id: UUID!): User

  # Policy Management
  policyDrafts(filter: PolicySearchFilter): [PolicyDraft!]!
  policyDraft(id: UUID!): PolicyDraft
  policyTemplates(organizationId: UUID!): [PolicyTemplate!]!
  policyTemplate(id: UUID!): PolicyTemplate

  # AI Features
  aiSuggestions(policyDraftId: UUID!, sectionId: String):
[AISuggestion!]!
  similarPolicies(policyDraftId: UUID!, limit: Int = 5):
[PolicyDraft!]!

  # Workflow
  workflows(policyDraftId: UUID): [Workflow!]!
  workflowTasks(assigneeId: UUID, status: String):
[WorkflowTask!]!

  # Analytics
  policyAnalytics(organizationId: UUID!, dateRange:
DateRangeInput): PolicyAnalytics!
}

# Mutations
type Mutation {
```

```
  # Policy Management
  createPolicyDraft(input: CreatePolicyDraftInput!):
PolicyDraft!
  updatePolicyDraft(id: UUID!, input: UpdatePolicyDraftInput!):
PolicyDraft!
  deletePolicyDraft(id: UUID!): Boolean!
  duplicatePolicyDraft(id: UUID!, title: String): PolicyDraft!

  # AI Integration
  generateAIContent(input: AIGenerationInput!):
AIGenerationResult!
  provideFeedback(interactionId: UUID!, score: Int!, comment:
String): Boolean!

  # Collaboration
  addCollaborator(policyDraftId: UUID!, userId: UUID!): Boolean!
  removeCollaborator(policyDraftId: UUID!, userId: UUID!):
Boolean!
  addComment(policyDraftId: UUID!, content: String!, sectionId:
String): Comment!

  # Workflow
  createWorkflow(policyDraftId: UUID!, definition: JSON!):
Workflow!
  updateWorkflowTask(taskId: UUID!, status: String!, comment:
String): WorkflowTask!
}

# Subscriptions
type Subscription {
  # Real-time collaboration
  policyDraftUpdated(id: UUID!): PolicyDraft!
  collaborationEvent(policyDraftId: UUID!): CollaborationEvent!

  # AI Generation
  aiGenerationProgress(contextId: UUID!): AIGenerationProgress!

  # Workflow
  workflowTaskAssigned(userId: UUID!): WorkflowTask!
}

# Additional Types
type AISuggestion {
  id: String!
  type: String!
  content: String!
  confidence: Float!
  reasoning: String
}

type AIGenerationResult {
  id: UUID!
```

```
  content: String!
  suggestions: [AISuggestion!]!
  metadata: JSON
}

type Comment {
  id: UUID!
  user: User!
  content: String!
  sectionId: String
  createdAt: DateTime!
}

type CollaborationEvent {
  type: String!
  user: User!
  data: JSON!
  timestamp: DateTime!
}

type AIGenerationProgress {
  status: String!
  progress: Float!
  estimatedTimeRemaining: Int
}

type PolicyAnalytics {
  totalDrafts: Int!
  draftsByStatus: JSON!
  averageCompletionTime: Float!
  aiUsageStats: JSON!
}
```

3.3 認証・認可設計

JWT Token Structure

```
interface AccessToken {
  // Standard Claims
  iss: string;           // Issuer
  sub: string;           // Subject (User ID)
  aud: string;           // Audience
  exp: number;           // Expiration Time
  iat: number;           // Issued At
  jti: string;           // JWT ID

  // Custom Claims
  email: string;         // User Email
  name: string;          // User Name
  org: string;           // Organization ID
```

```
  role: UserRole;         // User Role
  permissions: string[]; // Fine-grained Permissions
  scope: string[];        // OAuth Scopes
}

interface RefreshToken {
  sub: string;            // User ID
  jti: string;            // Token ID
  exp: number;            // Expiration (longer than access
token)
  type: 'refresh';        // Token Type
}
```

Permission System

```
// Permission Structure
interface Permission {
  resource: string;       // e.g., "policy_draft", "template",
"user"
  action: string;         // e.g., "create", "read", "update",
"delete"
  conditions?: {          // Optional conditions
    owner?: boolean;      // Only if user is owner
    organization?: string; // Only within specific org
    status?: string[];    // Only for specific statuses
  };
}

// Role Definitions
const ROLES = {
  ADMIN: {
    name: 'Administrator',
    permissions: [
      { resource: '*', action: '*' }, // Full access
    ]
  },
  MANAGER: {
    name: 'Manager',
    permissions: [
      { resource: 'policy_draft', action: '*' },
      { resource: 'template', action: '*' },
      { resource: 'user', action: 'read' },
      { resource: 'workflow', action: '*' },
    ]
  },
  EDITOR: {
    name: 'Editor',
    permissions: [
      { resource: 'policy_draft', action: 'create' },
      { resource: 'policy_draft', action: 'read' },
```

```
      { resource: 'policy_draft', action: 'update', conditions:
{ owner: true } },
      { resource: 'template', action: 'read' },
      { resource: 'ai_interaction', action: '*' },
    ]
  },
  VIEWER: {
    name: 'Viewer',
    permissions: [
      { resource: 'policy_draft', action: 'read' },
      { resource: 'template', action: 'read' },
    ]
  }
};

// Permission Checker
class PermissionChecker {
  static async checkPermission(
    user: User,
    resource: string,
    action: string,
    context?: any
  ): Promise<boolean> {
    const userPermissions = await this.getUserPermissions(user);

    for (const permission of userPermissions) {
      if (this.matchesPermission(permission, resource, action,
context)) {
        return true;
      }
    }

    return false;
  }

  private static matchesPermission(
    permission: Permission,
    resource: string,
    action: string,
    context?: any
  ): boolean {
    // Check resource match
    if (permission.resource !== '*' && permission.resource !==
resource) {
      return false;
    }

    // Check action match
    if (permission.action !== '*' && permission.action !==
action) {
      return false;
    }
```

```
    // Check conditions
    if (permission.conditions) {
      return this.checkConditions(permission.conditions,
context);
    }

    return true;
  }

  private static checkConditions(conditions: any, context:
any): boolean {
    if (conditions.owner && context?.userId !==
context?.resourceOwnerId) {
      return false;
    }

    if (conditions.organization && context?.organizationId !==
conditions.organization) {
      return false;
    }

    if (conditions.status && !
conditions.status.includes(context?.resourceStatus)) {
      return false;
    }

    return true;
  }
}
```

Phase 4: フロントエンド・UI設計

4.1 コンポーネント設計

アトミックデザイン原則の採用

```
Atoms (原子)
├── Button
├── Input
├── Icon
├── Typography
├── Spinner
└── Badge

Molecules (分子)
├── SearchBox (Input + Icon + Button)
├── FormField (Label + Input + ErrorMessage)
```

```
├── UserAvatar (Image + Badge)
├── ProgressBar (Bar + Typography)
└── Notification (Icon + Typography + Button)

Organisms (有機体)
├── Header (Logo + Navigation + UserMenu)
├── Sidebar (Navigation + UserInfo)
├── PolicyEditor (Monaco + Toolbar + StatusBar)
├── AIChat (MessageList + InputArea + Controls)
├── DocumentTree (TreeView + SearchBox + Actions)
└── WorkflowPanel (TaskList + ProgressBar + Actions)

Templates (テンプレート)
├── DashboardLayout
├── EditorLayout
├── SettingsLayout
└── AuthLayout

Pages (ページ)
├── Dashboard
├── PolicyEditor
├── TemplateLibrary
├── UserManagement
└── Settings
```

コンポーネント実装例

```tsx
// Atoms - Button Component
interface ButtonProps {
  variant: 'primary' | 'secondary' | 'danger' | 'ghost';
  size: 'sm' | 'md' | 'lg';
  disabled?: boolean;
  loading?: boolean;
  icon?: React.ReactNode;
  children: React.ReactNode;
  onClick?: () => void;
}

export const Button: React.FC<ButtonProps> = ({
  variant,
  size,
  disabled,
  loading,
  icon,
  children,
  onClick,
  ...props
}) => {
  const baseClasses = 'inline-flex items-center justify-center
font-medium rounded-md transition-colors';
```

```
  const variantClasses = {
    primary: 'bg-blue-600 text-white hover:bg-blue-700
focus:ring-blue-500',
    secondary: 'bg-gray-200 text-gray-900 hover:bg-gray-300
focus:ring-gray-500',
    danger: 'bg-red-600 text-white hover:bg-red-700 focus:ring-
red-500',
    ghost: 'text-gray-700 hover:bg-gray-100 focus:ring-gray-500'
  };
  const sizeClasses = {
    sm: 'px-3 py-1.5 text-sm',
    md: 'px-4 py-2 text-base',
    lg: 'px-6 py-3 text-lg'
  };

  return (
    <button
      className={cn(
        baseClasses,
        variantClasses[variant],
        sizeClasses[size],
        disabled && 'opacity-50 cursor-not-allowed'
      )}
      disabled={disabled || loading}
      onClick={onClick}
      {...props}
    >
      {loading && <Spinner className="mr-2" size="sm" />}
      {icon && !loading && <span className="mr-2">{icon}</span>}
      {children}
    </button>
  );
};

// Organisms - Policy Editor Component
interface PolicyEditorProps {
  policyDraft: PolicyDraft;
  onContentChange: (content: string) => void;
  onSave: () => void;
  onAIAssist: (prompt: string) => void;
  isLoading?: boolean;
}

export const PolicyEditor: React.FC<PolicyEditorProps> = ({
  policyDraft,
  onContentChange,
  onSave,
  onAIAssist,
  isLoading
}) => {
  const [editorContent, setEditorContent] =
useState(policyDraft.content);
```

```
  const [aiPanelOpen, setAiPanelOpen] = useState(false);
  const editorRef =
useRef<monaco.editor.IStandaloneCodeEditor>();

  const handleEditorMount = (editor:
monaco.editor.IStandaloneCodeEditor) => {
    editorRef.current = editor;

    // Custom language for policy documents
    monaco.languages.register({ id: 'policy-lang' });
    monaco.languages.setMonarchTokensProvider('policy-lang', {
      tokenizer: {
        root: [
          [/第\d+条/, 'article-number'],
          [/\d+\.\s/, 'section-number'],
          [/（[^）]+）/, 'parenthetical'],
          [/「[^」]+」/, 'quoted-text'],
        ]
      }
    });

    // Custom theme
    monaco.editor.defineTheme('policy-theme', {
      base: 'vs',
      inherit: true,
      rules: [
        { token: 'article-number', foreground: '0066cc',
fontStyle: 'bold' },
        { token: 'section-number', foreground: '008800' },
        { token: 'parenthetical', foreground: '666666',
fontStyle: 'italic' },
        { token: 'quoted-text', foreground: 'cc6600' },
      ],
      colors: {
        'editor.background': '#fafafa',
        'editor.lineHighlightBackground': '#f0f8ff',
      }
    });

    editor.updateOptions({
      theme: 'policy-theme',
      language: 'policy-lang',
      wordWrap: 'on',
      lineNumbers: 'on',
      minimap: { enabled: false },
      scrollBeyondLastLine: false,
    });
  };

  const handleContentChange = (value: string | undefined) => {
    if (value !== undefined) {
      setEditorContent(value);
```

```
      onContentChange(value);
    }
  };

  const insertAIContent = (content: string) => {
    if (editorRef.current) {
      const selection = editorRef.current.getSelection();
      const range = selection || new monaco.Range(1, 1, 1, 1);

      editorRef.current.executeEdits('ai-insert', [
        {
          range,
          text: content,
          forceMoveMarkers: true,
        }
      ]);
    }
  };

  return (
    <div className="flex h-full">
      {/* Main Editor */}
      <div className="flex-1 flex flex-col">
        {/* Toolbar */}
        <div className="border-b bg-white px-4 py-2 flex items-
center justify-between">
          <div className="flex items-center space-x-2">
            <Button
              variant="ghost"
              size="sm"
              icon={<SaveIcon />}
              onClick={onSave}
              disabled={isLoading}
            >
              保存
            </Button>
            <Button
              variant="ghost"
              size="sm"
              icon={<AIIcon />}
              onClick={() => setAiPanelOpen(!aiPanelOpen)}
            >
              AI アシスト
            </Button>
            <div className="h-4 w-px bg-gray-300" />
            <Button variant="ghost" size="sm" icon={<UndoIcon /
>}>
              元に戻す
            </Button>
            <Button variant="ghost" size="sm" icon={<RedoIcon /
>}>
              やり直し
```

```
            </Button>
          </div>

          <div className="flex items-center space-x-2">
            <Badge variant="secondary">{policyDraft.status}</
Badge>
            <Typography variant="sm" color="gray">
              最終更新: {formatDate(policyDraft.updatedAt)}
            </Typography>
          </div>
        </div>

        {/* Monaco Editor */}
        <div className="flex-1">
          <MonacoEditor
            height="100%"
            value={editorContent}
            onChange={handleContentChange}
            onMount={handleEditorMount}
            options={{
              automaticLayout: true,
              scrollBeyondLastLine: false,
              wordWrap: 'on',
              lineNumbers: 'on',
              minimap: { enabled: false },
            }}
          />
        </div>

        {/* Status Bar */}
        <div className="border-t bg-gray-50 px-4 py-2 flex
items-center justify-between text-sm text-gray-600">
          <div className="flex items-center space-x-4">
            <span>行:
{editorRef.current?.getPosition()?.lineNumber || 1}</span>
            <span>列: {editorRef.current?.getPosition()?.column
|| 1}</span>
            <span>文字数: {editorContent.length}</span>
          </div>
          <div className="flex items-center space-x-2">
            {isLoading && <Spinner size="sm" />}
            <span>Policy Language</span>
          </div>
        </div>
      </div>

      {/* AI Assistant Panel */}
      {aiPanelOpen && (
        <div className="w-80 border-l bg-white flex flex-col">
          <AIAssistantPanel
            policyDraft={policyDraft}
            onContentGenerated={insertAIContent}
```

```
            onClose={() => setAiPanelOpen(false)}
          />
        </div>
      )}
    </div>
  );
};

// AI Assistant Panel Component
interface AIAssistantPanelProps {
  policyDraft: PolicyDraft;
  onContentGenerated: (content: string) => void;
  onClose: () => void;
}

const AIAssistantPanel: React.FC<AIAssistantPanelProps> = ({
  policyDraft,
  onContentGenerated,
  onClose
}) => {
  const [messages, setMessages] = useState<ChatMessage[]>([]);
  const [inputValue, setInputValue] = useState('');
  const [isGenerating, setIsGenerating] = useState(false);

  const handleSendMessage = async () => {
    if (!inputValue.trim()) return;

    const userMessage: ChatMessage = {
      id: generateId(),
      role: 'user',
      content: inputValue,
      timestamp: new Date(),
    };

    setMessages(prev => [...prev, userMessage]);
    setInputValue('');
    setIsGenerating(true);

    try {
      const response = await aiService.generateContent({
        policyDraftId: policyDraft.id,
        prompt: inputValue,
        modelType: 'GPT_4O',
        contextType: 'generation',
      });

      const aiMessage: ChatMessage = {
        id: generateId(),
        role: 'assistant',
        content: response.content,
        timestamp: new Date(),
        suggestions: response.suggestions,
```

```
      };

      setMessages(prev => [...prev, aiMessage]);
    } catch (error) {
      console.error('AI generation failed:', error);
      // Handle error
    } finally {
      setIsGenerating(false);
    }
  };

  return (
    <div className="flex flex-col h-full">
      {/* Header */}
      <div className="border-b px-4 py-3 flex items-center
justify-between">
        <Typography variant="lg" weight="semibold">
          AI アシスタント
        </Typography>
        <Button variant="ghost" size="sm" onClick={onClose}>
          <CloseIcon />
        </Button>
      </div>

      {/* Chat Messages */}
      <div className="flex-1 overflow-y-auto p-4 space-y-4">
        {messages.map(message => (
          <ChatMessage
            key={message.id}
            message={message}
            onInsertContent={onContentGenerated}
          />
        ))}
        {isGenerating && (
          <div className="flex items-center space-x-2 text-
gray-500">
            <Spinner size="sm" />
            <span>生成中...</span>
          </div>
        )}
      </div>

      {/* Input Area */}
      <div className="border-t p-4">
        <div className="flex space-x-2">
          <Input
            value={inputValue}
            onChange={(e) => setInputValue(e.target.value)}
            placeholder="AIに質問や指示を入力..."
            onKeyPress={(e) => e.key === 'Enter' &&
handleSendMessage()}
            disabled={isGenerating}
```

```
            />
            <Button
              variant="primary"
              size="sm"
              onClick={handleSendMessage}
              disabled={!inputValue.trim() || isGenerating}
            >
              送信
            </Button>
          </div>

          {/* Quick Actions */}
          <div className="mt-3 flex flex-wrap gap-2">
            <Button
              variant="ghost"
              size="sm"
              onClick={() => setInputValue('この条項をより明確に書き直してください')}
            >
              条項の改善
            </Button>
            <Button
              variant="ghost"
              size="sm"
              onClick={() => setInputValue('類似の条項例を教えてください')}
            >
              類似条項検索
            </Button>
            <Button
              variant="ghost"
              size="sm"
              onClick={() => setInputValue('法的リスクをチェックしてください')}
            >
              リスクチェック
            </Button>
          </div>
        </div>
      </div>
    );
};
```

4.2 状態管理設計

Zustand Store Architecture

```
// Store Types
interface AppState {
```

```
  // User State
  user: UserState;

  // Policy Draft State
  policyDrafts: PolicyDraftState;

  // AI State
  ai: AIState;

  // UI State
  ui: UIState;

  // Workflow State
  workflow: WorkflowState;
}

// User State Management
interface UserState {
  currentUser: User | null;
  permissions: Permission[];
  organizations: Organization[];
  isAuthenticated: boolean;
  isLoading: boolean;
}

interface UserActions {
  login: (credentials: LoginCredentials) => Promise<void>;
  logout: () => void;
  updateProfile: (profile: Partial<User>) => Promise<void>;
  switchOrganization: (orgId: string) => Promise<void>;
}

const useUserStore = create<UserState & UserActions>((set, get)
=> ({
  // Initial State
  currentUser: null,
  permissions: [],
  organizations: [],
  isAuthenticated: false,
  isLoading: false,

  // Actions
  login: async (credentials) => {
    set({ isLoading: true });
    try {
      const response = await authService.login(credentials);
      const user = await userService.getCurrentUser();
      const permissions = await userService.getPermissions();

      set({
        currentUser: user,
        permissions,
```

```
        isAuthenticated: true,
        isLoading: false,
      });
    } catch (error) {
      set({ isLoading: false });
      throw error;
    }
  },

  logout: () => {
    authService.logout();
    set({
      currentUser: null,
      permissions: [],
      isAuthenticated: false,
    });
  },

  updateProfile: async (profile) => {
    const currentUser = get().currentUser;
    if (!currentUser) return;

    const updatedUser = await
userService.updateProfile(currentUser.id, profile);
    set({ currentUser: updatedUser });
  },

  switchOrganization: async (orgId) => {
    set({ isLoading: true });
    try {
      await userService.switchOrganization(orgId);
      const user = await userService.getCurrentUser();
      const permissions = await userService.getPermissions();

      set({
        currentUser: user,
        permissions,
        isLoading: false,
      });
    } catch (error) {
      set({ isLoading: false });
      throw error;
    }
  },
}));

// Policy Draft State Management
interface PolicyDraftState {
  drafts: Record<string, PolicyDraft>;
  currentDraftId: string | null;
  isLoading: boolean;
  searchResults: PolicyDraft[];
```

```
  filters: PolicySearchFilter;
}

interface PolicyDraftActions {
  loadDrafts: (filter?: PolicySearchFilter) => Promise<void>;
  loadDraft: (id: string) => Promise<void>;
  createDraft: (input: CreatePolicyDraftInput) =>
Promise<PolicyDraft>;
  updateDraft: (id: string, input: UpdatePolicyDraftInput) =>
Promise<void>;
  deleteDraft: (id: string) => Promise<void>;
  setCurrentDraft: (id: string | null) => void;
  searchDrafts: (query: string) => Promise<void>;
  updateFilters: (filters: Partial<PolicySearchFilter>) => void;
}

const usePolicyDraftStore = create<PolicyDraftState &
PolicyDraftActions>((set, get) => ({
  // Initial State
  drafts: {},
  currentDraftId: null,
  isLoading: false,
  searchResults: [],
  filters: {},

  // Actions
  loadDrafts: async (filter) => {
    set({ isLoading: true });
    try {
      const drafts = await policyService.getDrafts(filter);
      const draftsMap = drafts.reduce((acc, draft) => {
        acc[draft.id] = draft;
        return acc;
      }, {} as Record<string, PolicyDraft>);

      set({
        drafts: draftsMap,
        isLoading: false,
      });
    } catch (error) {
      set({ isLoading: false });
      throw error;
    }
  },

  loadDraft: async (id) => {
    set({ isLoading: true });
    try {
      const draft = await policyService.getDraft(id);
      set(state => ({
        drafts: { ...state.drafts, [id]: draft },
        isLoading: false,
```

```
      }));
    } catch (error) {
      set({ isLoading: false });
      throw error;
    }
  },

  createDraft: async (input) => {
    const draft = await policyService.createDraft(input);
    set(state => ({
      drafts: { ...state.drafts, [draft.id]: draft },
      currentDraftId: draft.id,
    }));
    return draft;
  },

  updateDraft: async (id, input) => {
    const updatedDraft = await policyService.updateDraft(id,
input);
    set(state => ({
      drafts: { ...state.drafts, [id]: updatedDraft },
    }));
  },

  deleteDraft: async (id) => {
    await policyService.deleteDraft(id);
    set(state => {
      const { [id]: deleted, ...remainingDrafts } =
state.drafts;
      return {
        drafts: remainingDrafts,
        currentDraftId: state.currentDraftId === id ? null :
state.currentDraftId,
      };
    });
  },

  setCurrentDraft: (id) => {
    set({ currentDraftId: id });
  },

  searchDrafts: async (query) => {
    const results = await policyService.searchDrafts(query);
    set({ searchResults: results });
  },

  updateFilters: (filters) => {
    set(state => ({
      filters: { ...state.filters, ...filters },
    }));
  },
}));
```

```
// AI State Management
interface AIState {
  contexts: Record<string, AIContext>;
  activeContextId: string | null;
  isGenerating: boolean;
  generationProgress: number;
  suggestions: AISuggestion[];
}

interface AIActions {
  initializeContext: (policyDraftId: string) =>
Promise<AIContext>;
  generateContent: (input: AIGenerationInput) =>
Promise<AIGenerationResult>;
  provideFeedback: (interactionId: string, feedback:
AIFeedback) => Promise<void>;
  getSuggestions: (policyDraftId: string, sectionId?: string)
=> Promise<void>;
  clearSuggestions: () => void;
}

const useAIStore = create<AIState & AIActions>((set, get) => ({
  // Initial State
  contexts: {},
  activeContextId: null,
  isGenerating: false,
  generationProgress: 0,
  suggestions: [],

  // Actions
  initializeContext: async (policyDraftId) => {
    const context = await
aiService.initializeContext(policyDraftId);
    set(state => ({
      contexts: { ...state.contexts, [context.id]: context },
      activeContextId: context.id,
    }));
    return context;
  },

  generateContent: async (input) => {
    set({ isGenerating: true, generationProgress: 0 });

    try {
      // Setup progress tracking
      const progressInterval = setInterval(() => {
        set(state => ({
          generationProgress: Math.min(state.generationProgress
+ 10, 90),
        }));
      }, 500);
```

```
      const result = await aiService.generateContent(input);

      clearInterval(progressInterval);
      set({
        isGenerating: false,
        generationProgress: 100,
      });

      // Reset progress after a delay
      setTimeout(() => {
        set({ generationProgress: 0 });
      }, 1000);

      return result;
    } catch (error) {
      set({
        isGenerating: false,
        generationProgress: 0,
      });
      throw error;
    }
  },

  provideFeedback: async (interactionId, feedback) => {
    await aiService.provideFeedback(interactionId, feedback);
  },

  getSuggestions: async (policyDraftId, sectionId) => {
    const suggestions = await
aiService.getSuggestions(policyDraftId, sectionId);
    set({ suggestions });
  },

  clearSuggestions: () => {
    set({ suggestions: [] });
  },
}));
```

4.3 UI/UX設計

デザインシステム

```
// Design Tokens
export const designTokens = {
  colors: {
    primary: {
      50: '#eff6ff',
      100: '#dbeafe',
      500: '#3b82f6',
```

```
      600: '#2563eb',
      700: '#1d4ed8',
      900: '#1e3a8a',
    },
    gray: {
      50: '#f9fafb',
      100: '#f3f4f6',
      200: '#e5e7eb',
      500: '#6b7280',
      700: '#374151',
      900: '#111827',
    },
    success: {
      50: '#ecfdf5',
      500: '#10b981',
      700: '#047857',
    },
    warning: {
      50: '#fffbeb',
      500: '#f59e0b',
      700: '#b45309',
    },
    error: {
      50: '#fef2f2',
      500: '#ef4444',
      700: '#b91c1c',
    },
  },

  typography: {
    fontFamily: {
      sans: ['Inter', 'system-ui', 'sans-serif'],
      mono: ['JetBrains Mono', 'monospace'],
    },
    fontSize: {
      xs: '0.75rem',
      sm: '0.875rem',
      base: '1rem',
      lg: '1.125rem',
      xl: '1.25rem',
      '2xl': '1.5rem',
      '3xl': '1.875rem',
    },
    fontWeight: {
      normal: '400',
      medium: '500',
      semibold: '600',
      bold: '700',
    },
  },

  spacing: {
```

```
    px: '1px',
    0: '0',
    1: '0.25rem',
    2: '0.5rem',
    3: '0.75rem',
    4: '1rem',
    5: '1.25rem',
    6: '1.5rem',
    8: '2rem',
    10: '2.5rem',
    12: '3rem',
    16: '4rem',
    20: '5rem',
    24: '6rem',
  },

  borderRadius: {
    none: '0',
    sm: '0.125rem',
    md: '0.375rem',
    lg: '0.5rem',
    xl: '0.75rem',
    full: '9999px',
  },

  shadows: {
    sm: '0 1px 2px 0 rgb(0 0 0 / 0.05)',
    md: '0 4px 6px -1px rgb(0 0 0 / 0.1)',
    lg: '0 10px 15px -3px rgb(0 0 0 / 0.1)',
    xl: '0 20px 25px -5px rgb(0 0 0 / 0.1)',
  },
};

// Layout Components
export const Layout = {
  Dashboard: ({ children }: { children: React.ReactNode }) => (
    <div className="min-h-screen bg-gray-50">
      <Header />
      <div className="flex">
        <Sidebar />
        <main className="flex-1 p-6">
          {children}
        </main>
      </div>
    </div>
  ),

  Editor: ({ children }: { children: React.ReactNode }) => (
    <div className="h-screen flex flex-col bg-white">
      <Header />
      <div className="flex-1 flex overflow-hidden">
        <DocumentSidebar />
```

```
        <main className="flex-1 flex flex-col">
          {children}
        </main>
      </div>
    </div>
  ),
};

// Responsive Design Breakpoints
export const breakpoints = {
  sm: '640px',
  md: '768px',
  lg: '1024px',
  xl: '1280px',
  '2xl': '1536px',
};

// Accessibility Guidelines
export const a11yGuidelines = {
  // Color contrast ratios
  colorContrast: {
    normal: 4.5, // WCAG AA
    large: 3,    // WCAG AA for large text
  },

  // Focus management
  focusManagement: {
    // Visible focus indicators
    focusRing: 'focus:ring-2 focus:ring-blue-500 focus:ring-
offset-2',
    // Skip links for keyboard navigation
    skipLink: 'sr-only focus:not-sr-only focus:absolute
focus:top-4 focus:left-4',
  },

  // ARIA labels and roles
  ariaLabels: {
    // Screen reader only text
    srOnly: 'sr-only',
    // Live regions for dynamic content
    liveRegion: 'aria-live="polite"',
    // Loading states
    loading: 'aria-busy="true"',
  },
};
```

Phase 5: 拡張性・保守性設計

5.1 プラグイン機構設計

プラグインアーキテクチャ

```
// Plugin Interface Definition
interface Plugin {
  id: string;
  name: string;
  version: string;
  description: string;
  author: string;
  dependencies?: string[];

  // Lifecycle hooks
  onActivate?(context: PluginContext): Promise<void>;
  onDeactivate?(): Promise<void>;

  // Extension points
  contributes?: PluginContributions;
}

interface PluginContributions {
  // UI Extensions
  commands?: CommandContribution[];
  menus?: MenuContribution[];
  views?: ViewContribution[];
  themes?: ThemeContribution[];

  // Functionality Extensions
  languages?: LanguageContribution[];
  aiProviders?: AIProviderContribution[];
  templates?: TemplateContribution[];
  workflows?: WorkflowContribution[];

  // Integration Extensions
  exporters?: ExporterContribution[];
  importers?: ImporterContribution[];
  validators?: ValidatorContribution[];
}

// Plugin Context
interface PluginContext {
  // Core APIs
  workspace: WorkspaceAPI;
  editor: EditorAPI;
  ai: AIAPI;
  ui: UIAPI;
```

```
  // Extension APIs
  commands: CommandAPI;
  storage: StorageAPI;
  http: HttpAPI;

  // Plugin metadata
  extensionPath: string;
  globalState: Memento;
  workspaceState: Memento;
}

// Plugin Manager
class PluginManager {
  private plugins = new Map<string, Plugin>();
  private activePlugins = new Set<string>();

  async loadPlugin(pluginPath: string): Promise<void> {
    const manifest = await this.loadManifest(pluginPath);
    const plugin = await this.instantiatePlugin(pluginPath,
manifest);

    // Validate dependencies
    await this.validateDependencies(plugin);

    this.plugins.set(plugin.id, plugin);
  }

  async activatePlugin(pluginId: string): Promise<void> {
    const plugin = this.plugins.get(pluginId);
    if (!plugin) throw new Error(`Plugin ${pluginId} not
found`);

    // Check dependencies
    await this.activateDependencies(plugin);

    // Create plugin context
    const context = this.createPluginContext(plugin);

    // Activate plugin
    if (plugin.onActivate) {
      await plugin.onActivate(context);
    }

    // Register contributions
    await this.registerContributions(plugin);

    this.activePlugins.add(pluginId);
  }

  async deactivatePlugin(pluginId: string): Promise<void> {
    const plugin = this.plugins.get(pluginId);
```

```
    if (!plugin) return;

    // Deactivate plugin
    if (plugin.onDeactivate) {
      await plugin.onDeactivate();
    }

    // Unregister contributions
    await this.unregisterContributions(plugin);

    this.activePlugins.delete(pluginId);
  }

  private async registerContributions(plugin: Plugin):
Promise<void> {
    if (!plugin.contributes) return;

    // Register commands
    if (plugin.contributes.commands) {
      for (const command of plugin.contributes.commands) {
        this.commandRegistry.register(command);
      }
    }

    // Register AI providers
    if (plugin.contributes.aiProviders) {
      for (const provider of plugin.contributes.aiProviders) {
        this.aiRegistry.register(provider);
      }
    }

    // Register other contributions...
  }
}

// Example Plugin Implementation
class CustomAIProviderPlugin implements Plugin {
  id = 'custom-ai-provider';
  name = 'Custom AI Provider';
  version = '1.0.0';
  description = 'Custom AI provider for specialized insurance
content';
  author = 'Insurance Corp';

  async onActivate(context: PluginContext): Promise<void> {
    // Register custom AI provider
    context.ai.registerProvider('custom-llm', new
CustomLLMProvider());

    // Register commands
    context.commands.register('custom-ai.generateClause', async
() => {
```

```
      const editor = context.editor.getActiveEditor();
      if (editor) {
        const content = await this.generateInsuranceClause();
        editor.insertText(content);
      }
    });
  }

  contributes = {
    commands: [
      {
        command: 'custom-ai.generateClause',
        title: 'Generate Insurance Clause',
        category: 'AI'
      }
    ],
    menus: [
      {
        command: 'custom-ai.generateClause',
        when: 'editorFocus',
        group: 'ai@1'
      }
    ],
    aiProviders: [
      {
        id: 'custom-llm',
        name: 'Custom Insurance LLM',
        description: 'Specialized LLM for insurance content',
        models: ['insurance-gpt-4', 'insurance-claude-3']
      }
    ]
  };

  private async generateInsuranceClause(): Promise<string> {
    // Custom AI generation logic
    return 'Generated insurance clause...';
  }
}
```

5.2 監視・ログ設計

包括的監視システム

```
// Metrics Collection
interface MetricsCollector {
  // Application Metrics
  recordAPILatency(endpoint: string, duration: number): void;
  recordDatabaseQuery(query: string, duration: number): void;
  recordAIGeneration(model: string, tokens: number, duration:
number): void;
```

```
  recordUserAction(action: string, userId: string): void;

  // Business Metrics
  recordPolicyDraftCreated(templateId: string, userId: string):
void;
  recordPolicyDraftCompleted(draftId: string, duration:
number): void;
  recordAIInteraction(type: string, success: boolean): void;
  recordWorkflowStep(workflowId: string, step: string): void;

  // Error Metrics
  recordError(error: Error, context: ErrorContext): void;
  recordAPIError(endpoint: string, statusCode: number): void;
}

class PrometheusMetricsCollector implements MetricsCollector {
  private registry = new prometheus.Registry();

  // Counters
  private apiRequestsTotal = new prometheus.Counter({
    name: 'api_requests_total',
    help: 'Total number of API requests',
    labelNames: ['method', 'endpoint', 'status_code'],
    registers: [this.registry]
  });

  private policyDraftsTotal = new prometheus.Counter({
    name: 'policy_drafts_total',
    help: 'Total number of policy drafts created',
    labelNames: ['template_id', 'organization_id'],
    registers: [this.registry]
  });

  // Histograms
  private apiDuration = new prometheus.Histogram({
    name: 'api_duration_seconds',
    help: 'API request duration in seconds',
    labelNames: ['method', 'endpoint'],
    buckets: [0.1, 0.5, 1, 2, 5, 10],
    registers: [this.registry]
  });

  private aiGenerationDuration = new prometheus.Histogram({
    name: 'ai_generation_duration_seconds',
    help: 'AI generation duration in seconds',
    labelNames: ['model', 'type'],
    buckets: [1, 5, 10, 30, 60, 120],
    registers: [this.registry]
  });

  // Gauges
  private activeUsers = new prometheus.Gauge({
```

```
    name: 'active_users',
    help: 'Number of active users',
    registers: [this.registry]
  });

  recordAPILatency(endpoint: string, duration: number): void {
    this.apiDuration.observe({ endpoint }, duration / 1000);
  }

  recordAIGeneration(model: string, tokens: number, duration:
number): void {
    this.aiGenerationDuration.observe({ model, type:
'generation' }, duration / 1000);
  }

  // Other implementations...
}

// Structured Logging
interface Logger {
  debug(message: string, meta?: LogMeta): void;
  info(message: string, meta?: LogMeta): void;
  warn(message: string, meta?: LogMeta): void;
  error(message: string, error?: Error, meta?: LogMeta): void;
}

interface LogMeta {
  userId?: string;
  organizationId?: string;
  requestId?: string;
  policyDraftId?: string;
  [key: string]: any;
}

class StructuredLogger implements Logger {
  constructor(private winston: winston.Logger) {}

  debug(message: string, meta: LogMeta = {}): void {
    this.winston.debug(message, {
      timestamp: new Date().toISOString(),
      level: 'debug',
      ...meta
    });
  }

  info(message: string, meta: LogMeta = {}): void {
    this.winston.info(message, {
      timestamp: new Date().toISOString(),
      level: 'info',
      ...meta
    });
  }
```

```
  warn(message: string, meta: LogMeta = {}): void {
    this.winston.warn(message, {
      timestamp: new Date().toISOString(),
      level: 'warn',
      ...meta
    });
  }

  error(message: string, error?: Error, meta: LogMeta = {}):
void {
    this.winston.error(message, {
      timestamp: new Date().toISOString(),
      level: 'error',
      error: error ? {
        name: error.name,
        message: error.message,
        stack: error.stack
      } : undefined,
      ...meta
    });
  }
}

// Health Check System
interface HealthCheck {
  name: string;
  check(): Promise<HealthStatus>;
}

interface HealthStatus {
  status: 'healthy' | 'unhealthy' | 'degraded';
  message?: string;
  details?: Record<string, any>;
}

class DatabaseHealthCheck implements HealthCheck {
  name = 'database';

  async check(): Promise<HealthStatus> {
    try {
      const start = Date.now();
      await this.database.query('SELECT 1');
      const duration = Date.now() - start;

      if (duration > 5000) {
        return {
          status: 'degraded',
          message: 'Database response time is slow',
          details: { responseTime: duration }
        };
      }
```

```
      return {
        status: 'healthy',
        details: { responseTime: duration }
      };
    } catch (error) {
      return {
        status: 'unhealthy',
        message: 'Database connection failed',
        details: { error: error.message }
      };
    }
  }
}

class AIServiceHealthCheck implements HealthCheck {
  name = 'ai-service';

  async check(): Promise<HealthStatus> {
    try {
      const providers = await
this.aiService.getAvailableProviders();
      const healthyProviders = providers.filter(p => p.status
=== 'healthy');

      if (healthyProviders.length === 0) {
        return {
          status: 'unhealthy',
          message: 'No AI providers available'
        };
      }

      if (healthyProviders.length < providers.length) {
        return {
          status: 'degraded',
          message: 'Some AI providers unavailable',
          details: {
            total: providers.length,
            healthy: healthyProviders.length
          }
        };
      }

      return {
        status: 'healthy',
        details: {
          providers: healthyProviders.map(p => p.name)
        }
      };
    } catch (error) {
      return {
        status: 'unhealthy',
```

```
        message: 'AI service check failed',
        details: { error: error.message }
      };
    }
  }
}
```

5.3 CI/CD設計

GitHub Actions Workflow

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  NODE_VERSION: '20'
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # Code Quality & Testing
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint code
        run: npm run lint

      - name: Type check
        run: npm run type-check

      - name: Run unit tests
        run: npm run test:unit
```

```yaml
      - name: Run integration tests
        run: npm run test:integration
        env:
          DATABASE_URL: postgresql://test:test@localhost:5432/
test
          REDIS_URL: redis://localhost:6379

      - name: Generate coverage report
        run: npm run test:coverage

      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v3
        with:
          file: ./coverage/lcov.info

  # Security Scanning
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Trivy vulnerability scanner
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          scan-ref: '.'
          format: 'sarif'
          output: 'trivy-results.sarif'

      - name: Upload Trivy scan results
        uses: github/codeql-action/upload-sarif@v2
        with:
          sarif_file: 'trivy-results.sarif'

      - name: Audit npm dependencies
        run: npm audit --audit-level=high

  # Build & Package
  build:
    needs: [quality, security]
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.meta.outputs.tags }}
      image-digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'
```

```yaml
      - name: Install dependencies
        run: npm ci

      - name: Build application
        run: npm run build

      - name: Log in to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=ref,event=branch
            type=ref,event=pr
            type=sha,prefix={{branch}}-
            type=raw,value=latest,enable={{is_default_branch}}

      - name: Build and push Docker image
        id: build
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # Deploy to Staging
  deploy-staging:
    if: github.ref == 'refs/heads/develop'
    needs: build
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to staging
        run: |
          echo "Deploying to staging environment"
          # Deployment logic here

      - name: Run E2E tests
        run: |
```

```
          echo "Running E2E tests against staging"
          # E2E test logic here

      - name: Notify deployment
        uses: 8398a7/action-slack@v3
        with:
          status: ${{ job.status }}
          channel: '#deployments'
          webhook_url: ${{ secrets.SLACK_WEBHOOK }}

  # Deploy to Production
  deploy-production:
    if: github.ref == 'refs/heads/main'
    needs: build
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to production
        run: |
          echo "Deploying to production environment"
          # Blue-green deployment logic here

      - name: Verify deployment
        run: |
          echo "Verifying production deployment"
          # Health check logic here

      - name: Notify deployment
        uses: 8398a7/action-slack@v3
        with:
          status: ${{ job.status }}
          channel: '#deployments'
          webhook_url: ${{ secrets.SLACK_WEBHOOK }}
```

Kubernetes Deployment Configuration

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: policy-editor-app
  labels:
    app: policy-editor
    version: v1
spec:
  replicas: 3
  selector:
    matchLabels:
```

```
      app: policy-editor
      version: v1
  template:
    metadata:
      labels:
        app: policy-editor
        version: v1
    spec:
      containers:
      - name: app
        image: ghcr.io/company/policy-editor:latest
        ports:
        - containerPort: 3000
        env:
        - name: NODE_ENV
          value: "production"
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: database-secret
              key: url
        - name: REDIS_URL
          valueFrom:
            secretKeyRef:
              name: redis-secret
              key: url
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 5
        volumeMounts:
        - name: config
          mountPath: /app/config
          readOnly: true
      volumes:
      - name: config
        configMap:
```

```
          name: policy-editor-config

---
apiVersion: v1
kind: Service
metadata:
  name: policy-editor-service
spec:
  selector:
    app: policy-editor
  ports:
  - protocol: TCP
    port: 80
    targetPort: 3000
  type: ClusterIP

---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: policy-editor-ingress
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/rate-limit: "100"
spec:
  tls:
  - hosts:
    - policy-editor.company.com
    secretName: policy-editor-tls
  rules:
  - host: policy-editor.company.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: policy-editor-service
            port:
              number: 80
```

5.4 テスト戦略

包括的テスト設計

```
// Unit Testing Strategy
describe('PolicyDraft Domain Model', () => {
  describe('updateContent', () => {
    it('should update content when user has edit permission',
```

```
() => {
      // Arrange
      const draft = new PolicyDraft(
        PolicyDraftId.generate(),
        OrganizationId.generate(),
        'Test Policy',
        new PolicyContent([]),
        PolicyStatus.DRAFT,
        new PolicyMetadata(),
        UserId.generate()
      );
      const newContent = new PolicyContent([
        new PolicySection('1', 'Section 1', 'Content', 1,
SectionType.CLAUSE)
      ]);

      // Act
      draft.updateContent(newContent, draft.createdBy);

      // Assert
      expect(draft.content).toEqual(newContent);
      expect(draft.getDomainEvents()).toHaveLength(1);
      expect(draft.getDomainEvents()
[0]).toBeInstanceOf(PolicyDraftUpdatedEvent);
    });

    it('should throw error when user lacks edit permission', ()
=> {
      // Arrange
      const draft = new PolicyDraft(/* ... */);
      const unauthorizedUser = UserId.generate();
      const newContent = new PolicyContent([]);

      // Act & Assert
      expect(() => {
        draft.updateContent(newContent, unauthorizedUser);
      }).toThrow('User does not have permission to edit this
draft');
    });
  });
});

// Integration Testing Strategy
describe('Policy Draft API Integration', () => {
  let app: INestApplication;
  let database: TestDatabase;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({
      imports: [AppModule],
    })
    .overrideProvider(DatabaseService)
```

```
    .useValue(new TestDatabaseService())
    .compile();

    app = moduleRef.createNestApplication();
    await app.init();

    database = new TestDatabase();
    await database.setup();
  });

  afterAll(async () => {
    await database.cleanup();
    await app.close();
  });

  describe('POST /api/policy-drafts', () => {
    it('should create policy draft with valid input', async ()
=> {
      // Arrange
      const user = await database.createUser();
      const template = await database.createTemplate();
      const createInput = {
        templateId: template.id,
        title: 'Test Policy',
        content: { sections: [] }
      };

      // Act
      const response = await request(app.getHttpServer())
        .post('/api/policy-drafts')
        .set('Authorization', `Bearer ${user.token}`)
        .send(createInput)
        .expect(201);

      // Assert
      expect(response.body).toMatchObject({
        id: expect.any(String),
        title: 'Test Policy',
        status: 'draft',
        createdBy: user.id
      });

      // Verify database state
      const draft = await
database.findPolicyDraft(response.body.id);
      expect(draft).toBeDefined();
    });
  });
});

// E2E Testing Strategy
describe('Policy Editor E2E', () => {
```

```
  let page: Page;
  let browser: Browser;

  beforeAll(async () => {
    browser = await chromium.launch();
  });

  afterAll(async () => {
    await browser.close();
  });

  beforeEach(async () => {
    page = await browser.newPage();
    await page.goto('http://localhost:3000');
  });

  afterEach(async () => {
    await page.close();
  });

  it('should create and edit policy draft', async () => {
    // Login
    await page.fill('[data-testid=email-input]',
'test@example.com');
    await page.fill('[data-testid=password-input]', 'password');
    await page.click('[data-testid=login-button]');

    // Navigate to editor
    await page.click('[data-testid=new-policy-button]');
    await page.waitForSelector('[data-testid=policy-editor]');

    // Edit content
    await page.fill('[data-testid=policy-title]', 'Test
Policy');
    await page.click('[data-testid=monaco-editor]');
    await page.keyboard.type('第1条（目的）\nこの約款は...');

    // Save
    await page.click('[data-testid=save-button]');
    await page.waitForSelector('[data-testid=save-success]');

    // Verify
    const title = await page.textContent('[data-testid=policy-
title]');
    expect(title).toBe('Test Policy');
  });

  it('should use AI assistance', async () => {
    // Setup
    await loginAsUser(page);
    await createPolicyDraft(page);
```

```
    // Open AI panel
    await page.click('[data-testid=ai-assist-button]');
    await page.waitForSelector('[data-testid=ai-panel]');

    // Send AI request
    await page.fill('[data-testid=ai-input]', '免責条項を生成してください');
    await page.click('[data-testid=ai-send-button]');

    // Wait for response
    await page.waitForSelector('[data-testid=ai-response]');

    // Insert content
    await page.click('[data-testid=ai-insert-button]');

    // Verify content inserted
    const editorContent = await page.textContent('[data-testid=monaco-editor]');
    expect(editorContent).toContain('免責');
  });
});

// Performance Testing Strategy
describe('Performance Tests', () => {
  it('should handle concurrent policy draft creation', async () => {
    const concurrentUsers = 50;
    const promises = Array.from({ length: concurrentUsers }, async (_, i) => {
      const user = await createTestUser(`user${i}`);
      const startTime = Date.now();

      const response = await request(app.getHttpServer())
        .post('/api/policy-drafts')
        .set('Authorization', `Bearer ${user.token}`)
        .send({
          title: `Policy ${i}`,
          content: { sections: [] }
        });

      const endTime = Date.now();
      return {
        userId: user.id,
        responseTime: endTime - startTime,
        success: response.status === 201
      };
    });

    const results = await Promise.all(promises);

    // Assert all requests succeeded
    expect(results.every(r => r.success)).toBe(true);
```

```
    // Assert 95th percentile response time < 2 seconds
    const responseTimes = results.map(r =>
r.responseTime).sort((a, b) => a - b);
    const p95 = responseTimes[Math.floor(responseTimes.length *
0.95)];
    expect(p95).toBeLessThan(2000);
  });
});
```

Phase 6: 実装ガイド・設計書作成

6.1 開発ガイドライン

コーディング規約

```
// TypeScript Coding Standards

// 1. Naming Conventions
// - PascalCase for classes, interfaces, types, enums
// - camelCase for variables, functions, methods
// - SCREAMING_SNAKE_CASE for constants
// - kebab-case for file names

// ✅ Good
class PolicyDraftService {
  private readonly MAX_DRAFT_SIZE = 1000000;

  async createDraft(input: CreatePolicyDraftInput):
Promise<PolicyDraft> {
    // Implementation
  }
}

interface CreatePolicyDraftInput {
  title: string;
  templateId: string;
  organizationId: string;
}

// ❌ Bad
class policyDraftService {
  private readonly maxDraftSize = 1000000;

  async CreateDraft(Input: createPolicyDraftInput):
Promise<PolicyDraft> {
    // Implementation
  }
```

```
}

// 2. Function and Method Design
// - Single Responsibility Principle
// - Pure functions when possible
// - Explicit return types
// - Descriptive parameter names

// ✅ Good
async function validatePolicyContent(
  content: PolicyContent,
  validationRules: ValidationRule[]
): Promise<ValidationResult> {
  const errors: ValidationError[] = [];

  for (const rule of validationRules) {
    const result = await rule.validate(content);
    if (!result.isValid) {
      errors.push(...result.errors);
    }
  }

  return {
    isValid: errors.length === 0,
    errors
  };
}

// ❌ Bad
async function validate(content: any, rules: any): Promise<any>
{
  // Implementation with unclear types and purpose
}

// 3. Error Handling
// - Use custom error classes
// - Provide meaningful error messages
// - Include context information

// ✅ Good
class PolicyDraftNotFoundError extends Error {
  constructor(
    public readonly draftId: string,
    public readonly organizationId: string
  ) {
    super(`Policy draft ${draftId} not found in organization $
{organizationId}`);
    this.name = 'PolicyDraftNotFoundError';
  }
}

class PolicyDraftService {
```

```
  async getDraft(id: string, organizationId: string):
Promise<PolicyDraft> {
    try {
      const draft = await this.repository.findById(id);

      if (!draft || draft.organizationId !== organizationId) {
        throw new PolicyDraftNotFoundError(id, organizationId);
      }

      return draft;
    } catch (error) {
      if (error instanceof PolicyDraftNotFoundError) {
        throw error;
      }

      throw new Error(`Failed to retrieve policy draft: $
{error.message}`);
    }
  }
}

// 4. Async/Await Best Practices
// - Always handle errors
// - Use Promise.all for parallel operations
// - Avoid nested async functions

// ✅ Good
async function processMultipleDrafts(draftIds: string[]):
Promise<ProcessResult[]> {
  try {
    const draftPromises = draftIds.map(id =>
this.processDraft(id));
    const results = await Promise.allSettled(draftPromises);

    return results.map((result, index) => ({
      draftId: draftIds[index],
      success: result.status === 'fulfilled',
      data: result.status === 'fulfilled' ? result.value : null,
      error: result.status === 'rejected' ? result.reason : null
    }));
  } catch (error) {
    throw new Error(`Failed to process drafts: ${error.message}
`);
  }
}

// 5. Type Safety
// - Use strict TypeScript configuration
// - Avoid 'any' type
// - Use union types and type guards

// ✅ Good
```

```typescript
type PolicyStatus = 'draft' | 'review' | 'approved' |
'published';

function isPolicyEditable(status: PolicyStatus): boolean {
  return status === 'draft' || status === 'review';
}

function isValidStatus(value: string): value is PolicyStatus {
  return ['draft', 'review', 'approved',
'published'].includes(value);
}

// 6. Documentation
// - Use JSDoc for public APIs
// - Include examples for complex functions
// - Document business logic

/**
 * Generates AI-assisted content for a policy section
 *
 * @param sectionId - The ID of the section to generate content
for
 * @param prompt - The user's prompt for content generation
 * @param options - Configuration options for AI generation
 * @returns Promise resolving to generated content and metadata
 *
 * @example
 * ```typescript
 * const result = await generateSectionContent(
 *   'section-1',
 *   'Generate a liability clause for auto insurance',
 *   { model: 'gpt-4', temperature: 0.7 }
 * );
 * console.log(result.content);
 * ```
 */
async function generateSectionContent(
  sectionId: string,
  prompt: string,
  options: AIGenerationOptions
): Promise<AIGenerationResult> {
  // Implementation
}
```

Git Workflow

```
# Branch Naming Convention
feature/TICKET-123-add-ai-suggestions
bugfix/TICKET-456-fix-editor-crash
hotfix/TICKET-789-security-patch
```

```
release/v1.2.0

# Commit Message Format
# <type>(<scope>): <subject>
#
# <body>
#
# <footer>

# Examples:
feat(editor): add AI-powered content suggestions

- Implement AI suggestion panel
- Add integration with OpenAI API
- Include user feedback mechanism

Closes #123

fix(auth): resolve JWT token expiration issue

- Update token refresh logic
- Add proper error handling for expired tokens
- Improve user experience during re-authentication

Fixes #456

# Development Workflow
git checkout develop
git pull origin develop
git checkout -b feature/TICKET-123-add-ai-suggestions

# Make changes and commit
git add .
git commit -m "feat(editor): add AI-powered content suggestions"

# Push and create PR
git push origin feature/TICKET-123-add-ai-suggestions
# Create Pull Request via GitHub UI

# Code Review Process
# 1. Automated checks must pass (CI/CD)
# 2. At least 2 reviewers must approve
# 3. All conversations must be resolved
# 4. Branch must be up-to-date with target branch

# Merge Process
# 1. Squash and merge for feature branches
# 2. Merge commit for release branches
# 3. Delete feature branch after merge
```

6.2 運用手順書

デプロイメント手順

```
 # Production Deployment Checklist

## Pre-deployment
# 1. Verify all tests pass
npm run test:all

# 2. Check security vulnerabilities
npm audit --audit-level=high

# 3. Update version
npm version patch  # or minor/major

# 4. Build and test Docker image
docker build -t policy-editor:latest .
docker run --rm policy-editor:latest npm run test:unit

# 5. Database migration (if needed)
kubectl apply -f k8s/migrations/

## Deployment
# 1. Deploy to staging first
kubectl apply -f k8s/staging/ --namespace=staging

# 2. Run smoke tests
npm run test:smoke -- --env=staging

# 3. Deploy to production (blue-green)
kubectl apply -f k8s/production/ --namespace=production

# 4. Verify health checks
kubectl get pods -n production
kubectl logs -f deployment/policy-editor-app -n production

# 5. Run production smoke tests
npm run test:smoke -- --env=production

## Post-deployment
# 1. Monitor metrics for 30 minutes
# 2. Check error rates and response times
# 3. Verify AI services are functioning
# 4. Update deployment documentation

## Rollback Procedure (if needed)
# 1. Identify previous stable version
kubectl rollout history deployment/policy-editor-app -n
production
```

```
# 2. Rollback to previous version
kubectl rollout undo deployment/policy-editor-app -n production

# 3. Verify rollback success
kubectl rollout status deployment/policy-editor-app -n
production
```

監視・アラート設定

```
# Prometheus Alert Rules
groups:
- name: policy-editor-alerts
  rules:
  # High Error Rate
  - alert: HighErrorRate
    expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.1
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"
      description: "Error rate is {{ $value }} errors per
second"

  # High Response Time
  - alert: HighResponseTime
    expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m])) > 2
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "High response time detected"
      description: "95th percentile response time is {{
$value }} seconds"

  # AI Service Down
  - alert: AIServiceDown
    expr: up{job="ai-service"} == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "AI service is down"
      description: "AI service has been down for more than 1
minute"

  # Database Connection Issues
  - alert: DatabaseConnectionHigh
    expr:
```

```
database_connections_active / database_connections_max > 0.8
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "High database connection usage"
      description: "Database connection usage is {{ $value }}%"

  # Disk Space Low
  - alert: DiskSpaceLow
    expr: (node_filesystem_avail_bytes /
node_filesystem_size_bytes) < 0.1
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Low disk space"
      description: "Disk space is {{ $value }}% full"

# Grafana Dashboard Configuration
{
  "dashboard": {
    "title": "Policy Editor Monitoring",
    "panels": [
      {
        "title": "Request Rate",
        "type": "graph",
        "targets": [
          {
            "expr": "rate(http_requests_total[5m])",
            "legendFormat": "{{method}} {{endpoint}}"
          }
        ]
      },
      {
        "title": "Response Time",
        "type": "graph",
        "targets": [
          {
            "expr": "histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m]))",
            "legendFormat": "95th percentile"
          },
          {
            "expr": "histogram_quantile(0.50,
rate(http_request_duration_seconds_bucket[5m]))",
            "legendFormat": "50th percentile"
          }
        ]
      },
      {
        "title": "AI Generation Metrics",
```

```
        "type": "graph",
        "targets": [
          {
            "expr": "rate(ai_generations_total[5m])",
            "legendFormat": "Generations per second"
          },
          {
            "expr": "histogram_quantile(0.95,
rate(ai_generation_duration_seconds_bucket[5m]))",
            "legendFormat": "Generation time (95th percentile)"
          }
        ]
      }
    ]
  }
}
```

トラブルシューティングガイド

```
# Troubleshooting Guide

## Common Issues

### 1. Application Won't Start

**Symptoms:**
- Container exits immediately
- Health check failures
- Connection refused errors

**Diagnosis:**
```bash
# Check container logs
kubectl logs deployment/policy-editor-app -n production

# Check pod status
kubectl describe pod <pod-name> -n production

# Check configuration
kubectl get configmap policy-editor-config -o yaml
```

Solutions: - Verify environment variables - Check database connectivity - Validate configuration files - Ensure required secrets exist

2. High Memory Usage

Symptoms: - OOMKilled pod restarts - Slow response times - Memory alerts firing
```

Diagnosis:

```
# Check memory usage
kubectl top pods -n production

# Check memory limits
kubectl describe pod <pod-name> -n production

# Analyze heap dumps (if available)
node --inspect app.js
```

Solutions: - Increase memory limits - Optimize database queries - Implement caching - Check for memory leaks

3. AI Service Timeouts

Symptoms: - AI generation requests failing - Timeout errors in logs - User complaints about AI features

Diagnosis:

```
# Check AI service logs
kubectl logs deployment/ai-service -n production

# Test AI endpoints
curl -X POST https://api.openai.com/v1/chat/completions \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "gpt-4", "messages": [{"role": "user",
"content": "test"}]}'
```

Solutions: - Increase timeout values - Implement retry logic - Switch to backup AI provider - Check API rate limits

4. Database Performance Issues

Symptoms: - Slow query responses - Database connection pool exhaustion - Lock wait timeouts

Diagnosis:

```
-- Check slow queries
SELECT query, mean_time, calls
FROM pg_stat_statements
ORDER BY mean_time DESC
```

```
LIMIT 10;

-- Check active connections
SELECT count(*) FROM pg_stat_activity;

-- Check locks
SELECT * FROM pg_locks WHERE NOT granted;
```

Solutions: - Optimize slow queries - Add missing indexes - Increase connection pool size - Implement query caching

Emergency Procedures

1. Complete Service Outage

1. **Immediate Response (0-5 minutes)**
2. Check overall system status
3. Verify infrastructure (K8s, database, external services)
4. Activate incident response team
5. **Assessment (5-15 minutes)**
6. Identify root cause
7. Estimate impact and recovery time
8. Communicate with stakeholders
9. **Recovery (15+ minutes)**
10. Implement fix or rollback
11. Verify service restoration
12. Monitor for stability

2. Data Corruption

1. **Stop writes immediately** `bash kubectl scale deployment policy-editor-app --replicas=0`
2. **Assess damage** `sql -- Check data integrity SELECT COUNT(*) FROM policy_drafts WHERE content IS NULL;`
3. **Restore from backup** `bash # Restore database from latest backup pg_restore -d policy_editor backup_file.sql`

4. **Verify restoration**

5. Run data validation scripts
6. Test critical user journeys
7. Gradually restore service

3. Security Incident

1. **Immediate containment**
2. Isolate affected systems
3. Revoke compromised credentials

4. Enable additional logging

5. **Investigation**

6. Collect forensic evidence
7. Analyze attack vectors

8. Assess data exposure

9. **Recovery**

10. Patch vulnerabilities
11. Update security measures

12. Restore services securely

13. **Post-incident**

14. Conduct security review
15. Update incident response procedures
16. Communicate with affected users ```

6.3 最終設計書サマリー

アーキテクチャ概要

本設計書では、メンテナンス性と拡張性を重視した保険約款ドラフト作成アプリケーションの包括的な技術設計を提示しました。

主要な設計決定:

1. **技術スタック**
2. フロントエンド: React + TypeScript + Monaco Editor
3. バックエンド: Node.js + NestJS + TypeScript

4. データベース: PostgreSQL + Redis + MinIO
5. AI統合: LangChain + 複数LLMプロバイダー

6. インフラ: Docker + Kubernetes + Terraform

7. **アーキテクチャパターン**

8. ヘキサゴナルアーキテクチャ（ポート&アダプター）
9. ドメイン駆動設計（DDD）
10. マイクロサービス分割

11. イベント駆動アーキテクチャ

12. **品質保証**

13. 包括的テスト戦略（Unit/Integration/E2E/Performance）
14. CI/CDパイプライン
15. 監視・ログシステム
16. セキュリティ多層防御

実装優先順位

Phase 1: 基盤構築（3-4ヶ月） - 認証・認可システム - 基本的なポリシードラフト管理 - Monaco Editorベースのエディター - 基本的なAI統合

Phase 2: コア機能（4-5ヶ月） - 高度なAI機能（RAG、複数プロバイダー） - ワークフロー管理 - リアルタイムコラボレーション - テンプレート管理

Phase 3: 拡張機能（3-4ヶ月） - プラグインシステム - 高度な分析・レポート - 外部システム連携 - モバイル対応

成功指標

技術指標: - システム可用性: 99.9%以上 - レスポンス時間: 95%のリクエストが3秒以内 - コードカバレッジ: 80%以上 - セキュリティ脆弱性: Critical/High 0件

ビジネス指標: - ユーザー満足度: 4.5/5以上 - 約款作成時間: 50%短縮 - AI機能利用率: 80%以上 - システム導入成功率: 95%以上

リスク軽減策

技術リスク: - プロトタイプによる技術検証 - 段階的実装・テスト - 複数技術選択肢の並行検討 - 専門家によるアーキテクチャレビュー

運用リスク: - 包括的監視・アラートシステム - 自動化されたデプロイメント - 災害復旧計画 - 定期的なセキュリティ監査

この設計書に基づいて実装を進めることで、保険業界の要求に応える高品質で拡張可能なアプリケーションの構築が可能になります。

設計書作成日: 2025年6月6日
設計対象: 保険約款ドラフト作成アプリケーション
設計方針: メンテナンス性・拡張性重視
想定利用者: 保険会社の約款作成担当者
システム規模: エンタープライズレベル